

version of the language. The new version has *let*. The new *let* gives block scope to JavaScript.

There are tons of programmatic conveniences like arrow functions, *for..of* constructs, string functions, function parameters with default values, variable parameters, destructuring and template strings.

The new features such as high-level languages include Promise and meta programming. With Promise, you can have actions completed in a deferred fashion. If you are familiar with *jQuery.Deferred*, then you know Promise. Meta programming is more a theme than a feature. Meta programming includes *Symbols*, *Reflect* and *Proxies*.

For those inclined towards functional programming, there's the new Math library, new data structures such as Maps, Sets, WeakMaps and WeakSets.

Support for object-oriented constructs is possible with fully defined *class* and *extends* keywords. With enhanced support of modules and facilities of import/export, you can load or pick only the required portions of a library. This is efficient and helps avoiding conflicts.

This article is just a broad view of what is going to be covered in detail in the coming months.

Language environment

Different browsers have varied levels of support for ES6 at this point. There are two ways to learn ES6. The first is by using the cloud editor (es6fiddle.net, see *References*) to try code samples. You can also type, evaluate and learn. The second way is by using translators such as Babel and Traceur, which translate ES6 code to ES5. The resulting JavaScript code is used in production to be sure your application works in the same way across all browsers.

Why move to ES6?

There are multiple reasons to move to ES6, depending on your role in an organisation and your interests. If you are a developer, you will find ways to write more elegant JavaScript that is easy to write and read. It is more maintainable and with less side effects. If you are an architect, it will help you make right choices when selecting JavaScript libraries and frameworks. You might be better off using pure JavaScript, if the new version eliminates the need for an additional framework. And on the server side, you can right away use JavaScript, new version without worrying about client browser support. If you suspect your clients may not use browsers that are of the latest versions, you can use JavaScript compilers to convert to the ES5 version, as most browsers support it.

Block scoping with *let* and *const*

Let us start learning the new features. To start with, block scoping is an easy-to-understand feature. You now can restrict a variable to block scope such as for a loop, *if-then* or *if-else* blocks. The variable is available even within sub-blocks.

```
1. for (let i = 0; i < 3; i++) {
```

```
2.   console.log('iterator ' + i);
3. }
4. console.log('iterator ' + i++); // error: i is not defined
```

Code 1: An example for *let*

The new keyword *const* can be used to define a constant. Note that *const* is more like *let* in relation to scope, than *var* (see Code 1, Code 2 and Code 3).

```
1. const version = 6;
2. console.log(version);
3. version = 7; // error: version is read-only
```

Code 2: An example for *const*

```
1. {
2.   const version = 6;
3.   console.log(version);
4. }
5. version = 7; // error: version is not defined
```

Code 3: An example for *const* with scope

Template strings

In JavaScript, string concatenation has been through a series of strings, variables and the *+* operator. This makes log message statements cumbersome and error-prone with double-quotes spread all over. Most programmers would have experienced missing a closing double-quote and the resulting weird behaviour of the program!

Note that the quote type used is backtick or back-quote. It is not the single quote.

```
1. let i = 1;
2. console.log(`iterator ${i}`); // print: iterator 1
```

There'll be more coverage on these topics in the coming months. 

References

- [1] ECMAScript 2015 Language Specification <http://www.ecma-international.org/publications/standards/Ecma-262.htm>
- [2] ECMAScript 2015 Language support across browsers <https://kangax.github.io/compat-table/es6/>
- [3] Try ES6 coding and learn through examples <http://www.es6fiddle.net/>
- [4] Babel JavaScript Compiler <https://babeljs.io/>
- [5] ECMAScript on GitHub <https://github.com/tc39>
- [6] Mozilla Developer Network - New features <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

By: Janardan Revuru

The author is passionate about Web technologies and takes a keen interest in the baby (JavaScript) that has survived for over two decades, against all odds.. He can be reached at janardan.revuru@gmail.com for career counselling and clarifications.